Pipelining

#16

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Fall 2020

Announcements

- OMETS!!!!!!!!11!!!!!!!!!1!!!1eleven!!!!

How can we make the CPU more efficient? …

Doing the laundry

Luis (me), Artur, Stephen, and Ray have one load of clothes to

- Wash



Washer takes 60 minutes

- Dry



Dryer takes 60 minutes

- Fold

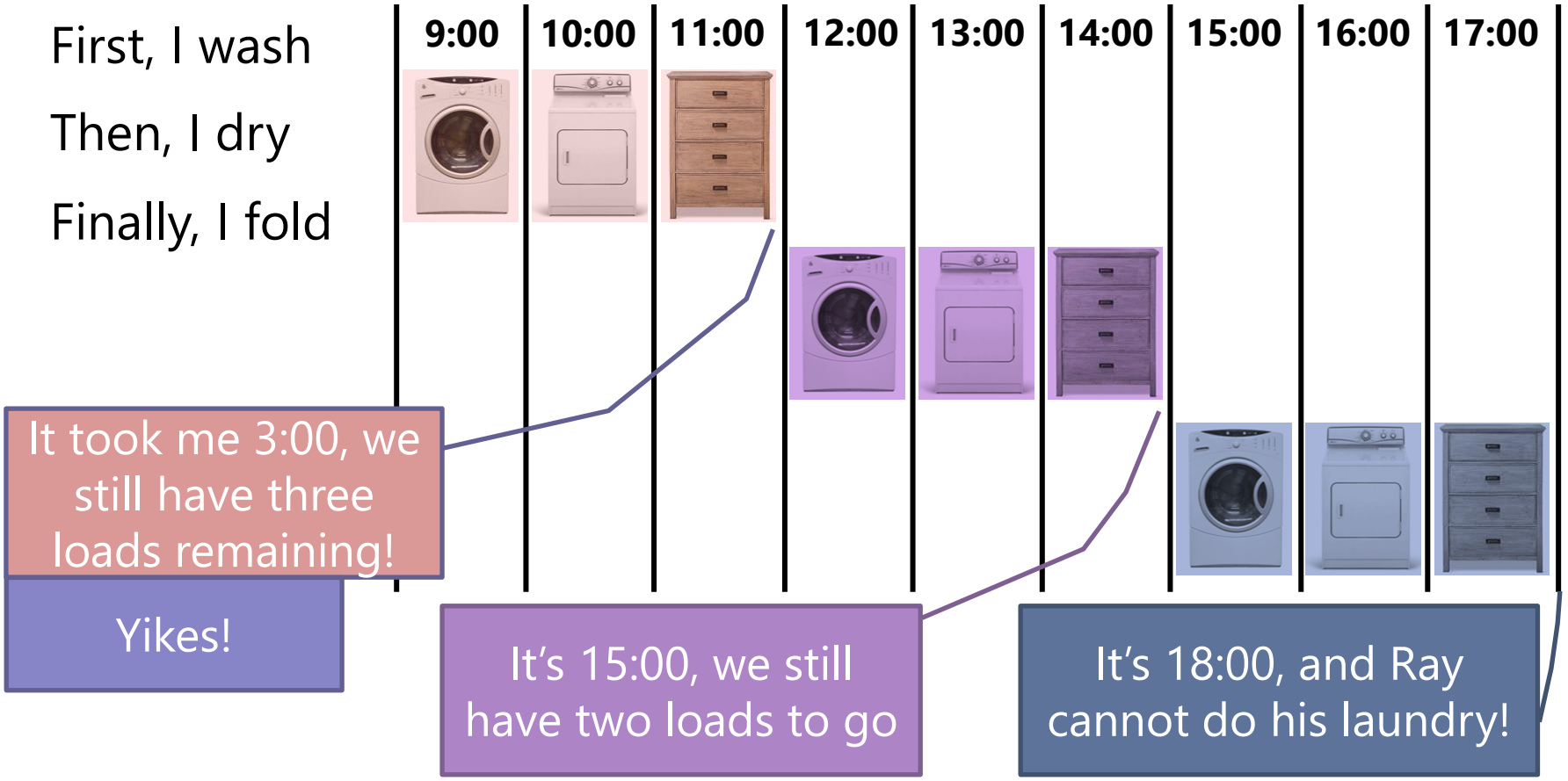


Folding takes 60 minutes

Where we live, we can only do laundry Saturday from 9:00 to 18:00!

Sequential laundry

- We have four loads of laundry to do (Luis, Artur, Stephen, and Ray)



How can we solve this?

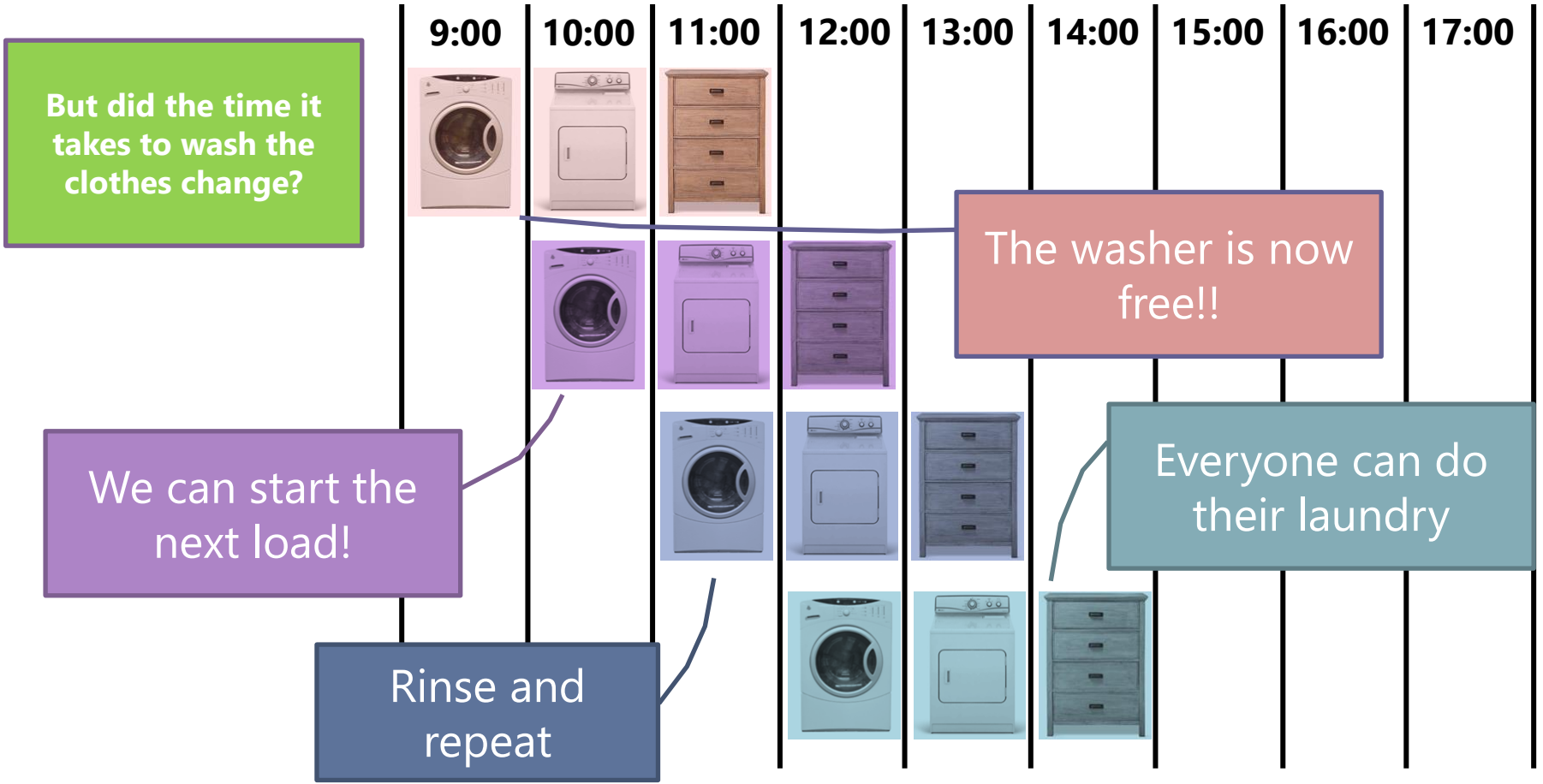
- Buy more machines!!!



- Or…

Pipelined laundry

- We have four loads of laundry to do (Luis, Artur, Stephen, and Ray)

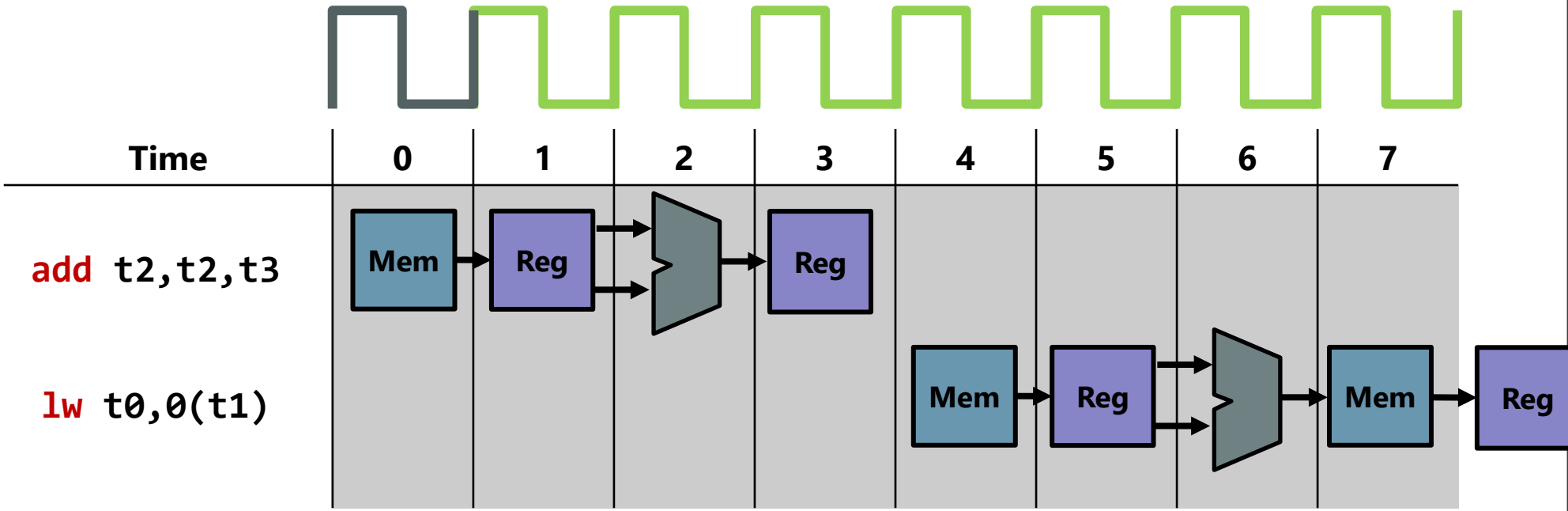


Upgrading the multi-cycle CPU

Let's apply the same concept to a multi-cycle CPU!

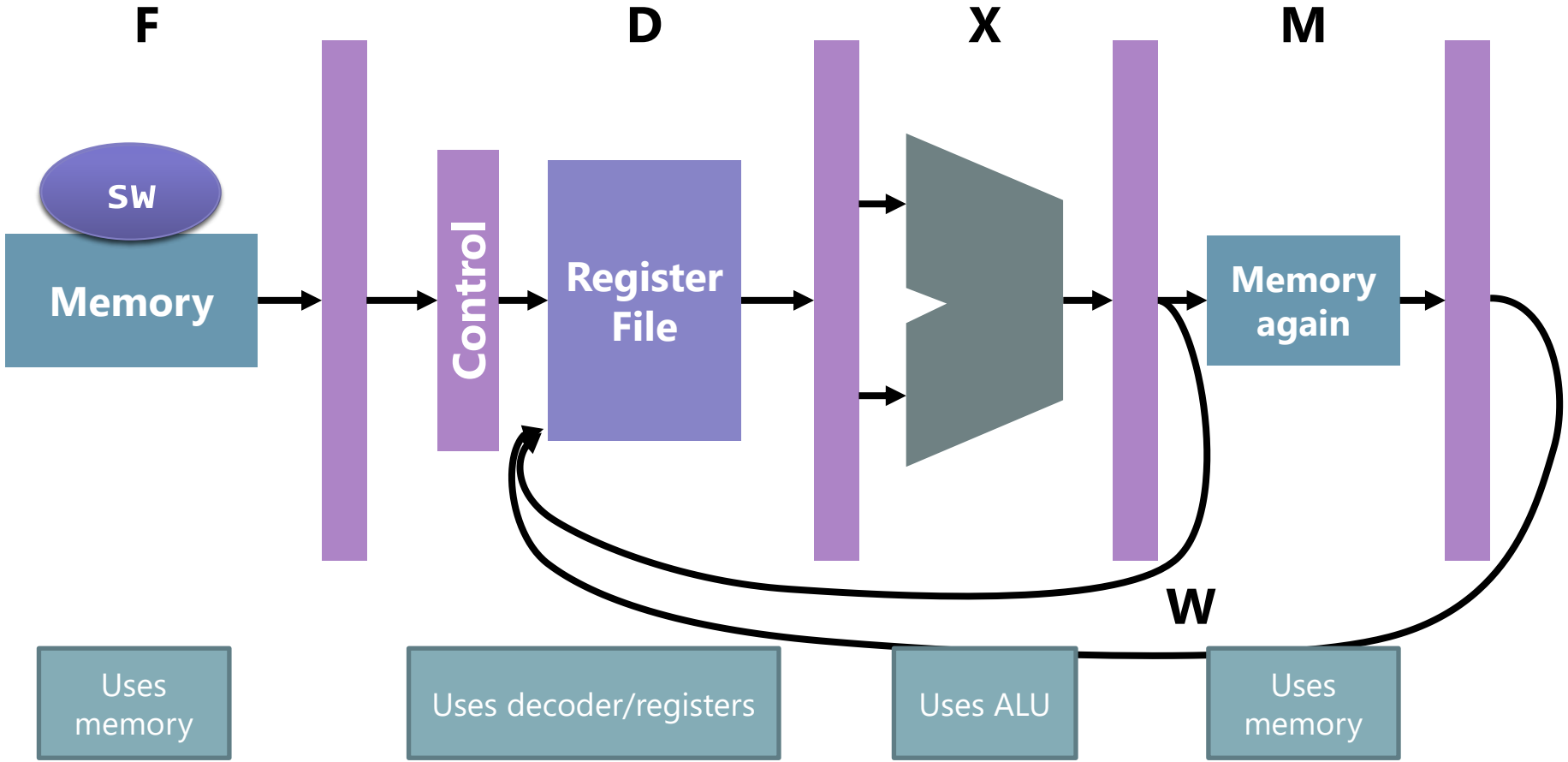
Keep the same clock

- Reuse resources with …



Lighting up the silicon (animated)

Executing instructions in a pipeline

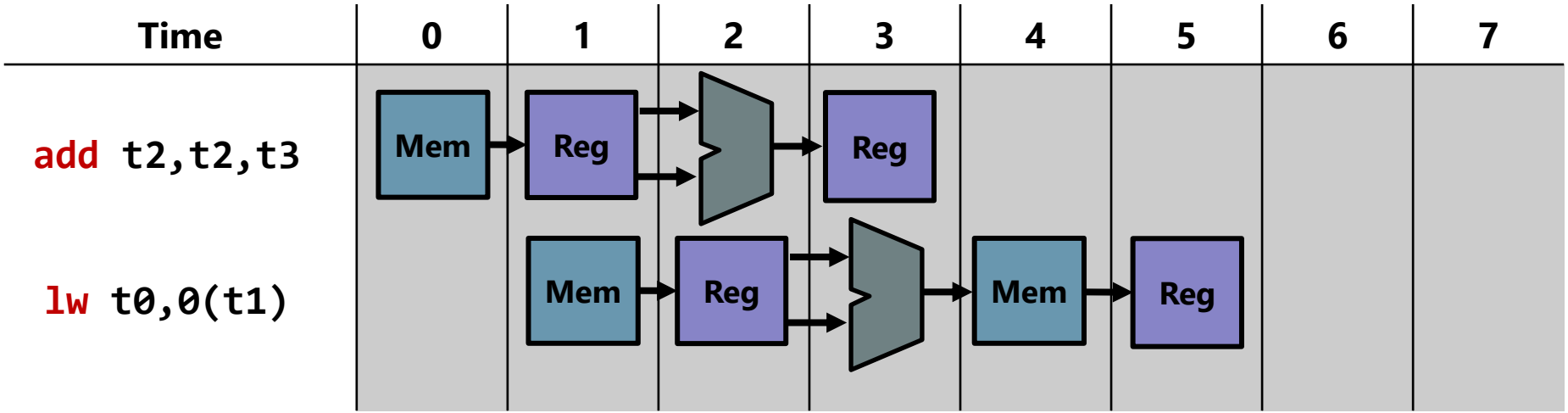


Pipeline vs Multi-cycle

Pipelining doesn't help **latency** of a single instruction!
It helps **throughput** of the entire workload!

Different tasks operating **different resources**
can execute simultaneously

More stages, more potential speedup (too many stages is not good!)



So how did we improve performance?

- Did we make any **individual instruction** faster?
  - No, the add still took 4 cycles… the lw took 5 cycles

- But the whole thing finished **faster.** right?

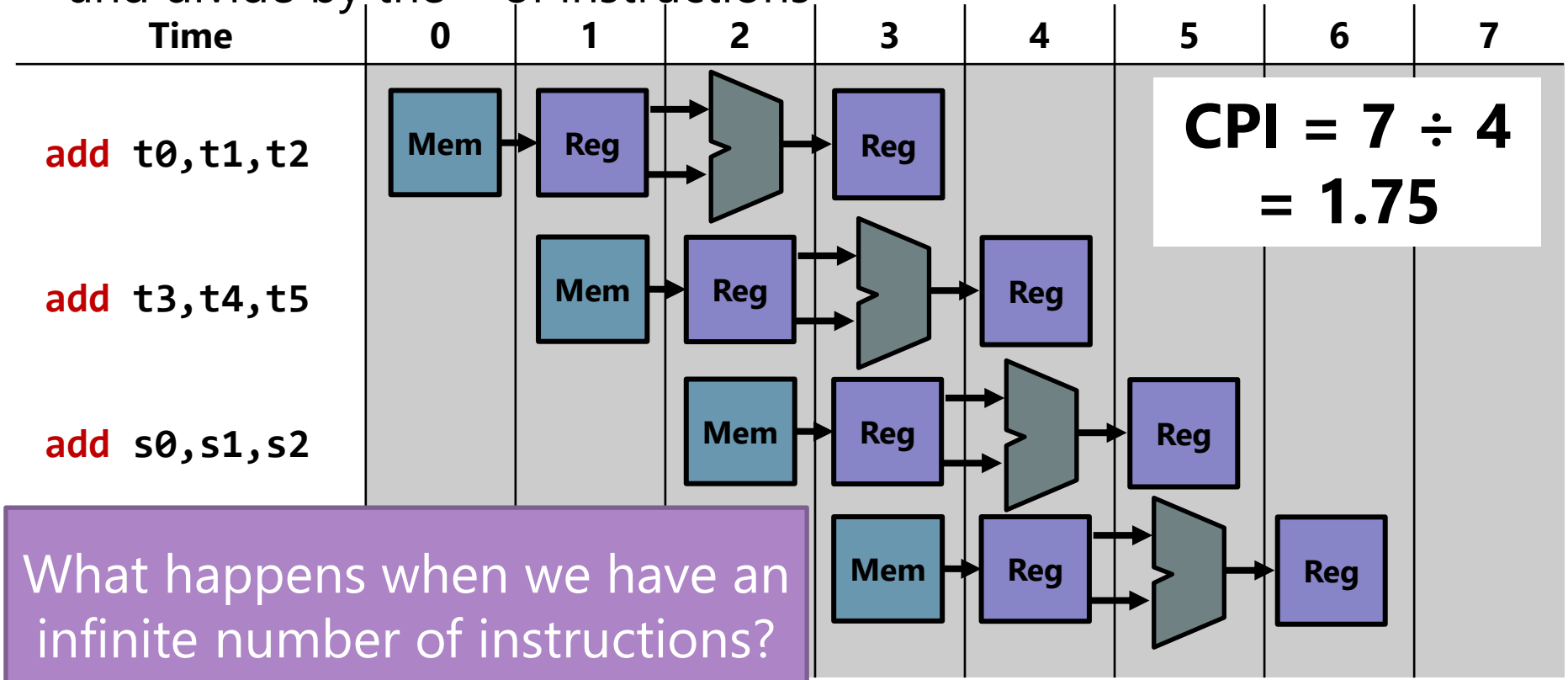
Yes, by overlapping the instructions,
we increased the **throughput.**

In any given clock cycle, we're
now doing **more work.**

With this we can get the CPI
down closer to 1.

The *average* CPI

- It's the average number of Cycles Per Instruction
  - For any program, we count the # of cycles
    - and divide by the # of instructions



What happens when we have an infinite number of instructions?

Pipeline (real-world) issues

Instructions are co-dependent ☹

- Sometimes, the next instruction cannot execute in the next cycle
  - We call those **pipeline hazards**.

- Hazards happen when for any reason an instruction is unable to advance (execute) in the pipeline

- We'll look at three types of hazards
  - **Structural hazards**
  - **Data hazards**
  - **Control hazards**

Structural hazards

Attempting to use the same resource two different ways simultaneously. E.g.:

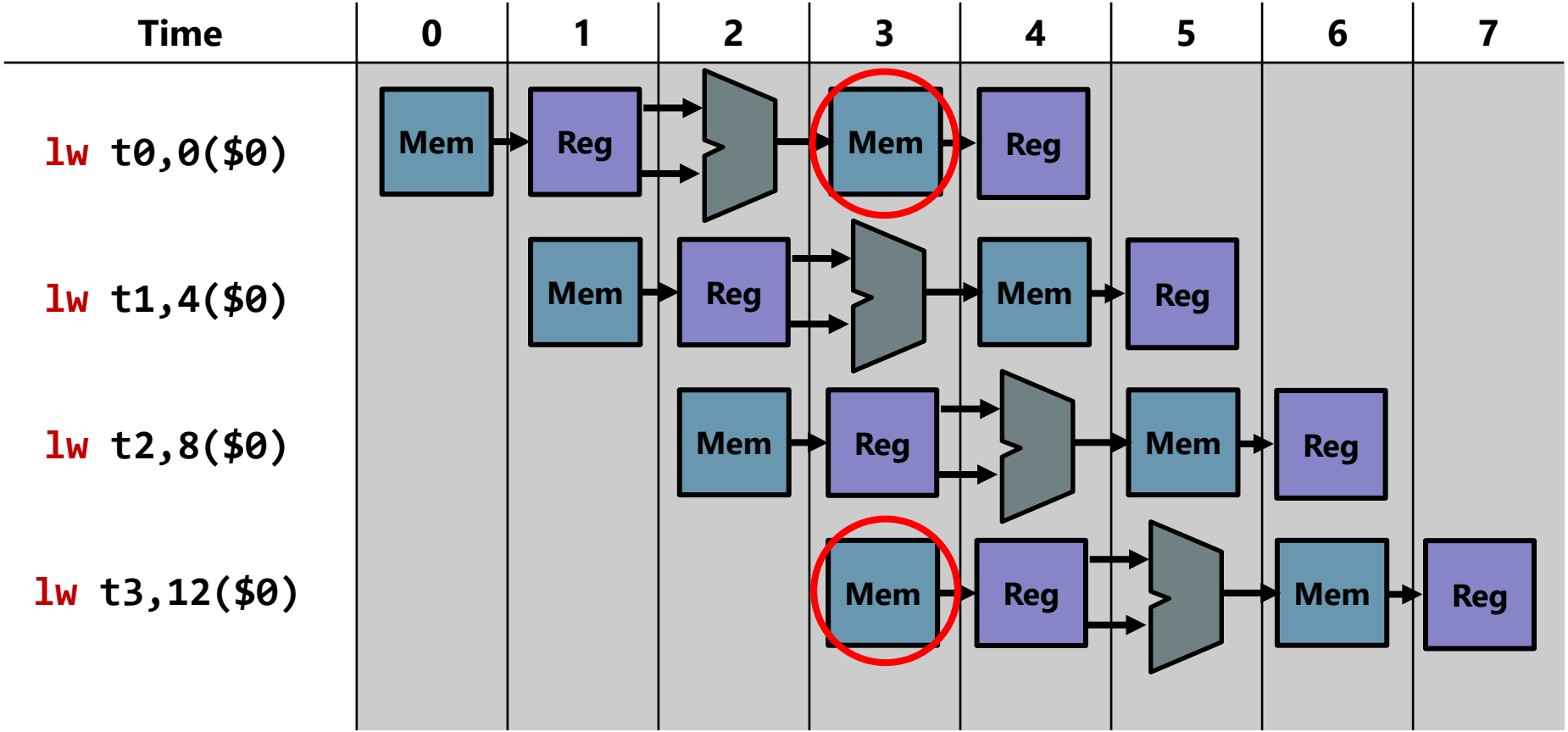
- You get home soaking wet and need to dry your clothes while someone is using the dryer

In a CPU with a single memory:

- Can we fetch a new instruction while reading a word from memory?
  - **NOP**e: structural hazard

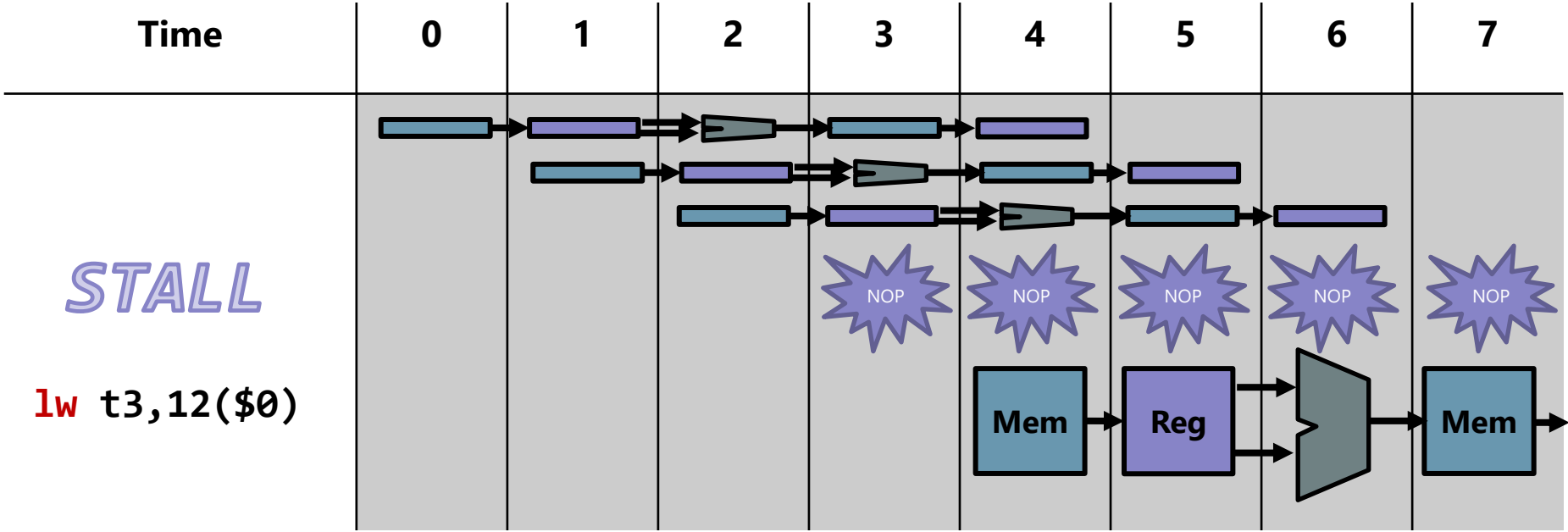
Structural hazards

- Two instructions using the **same hardware** at the same time



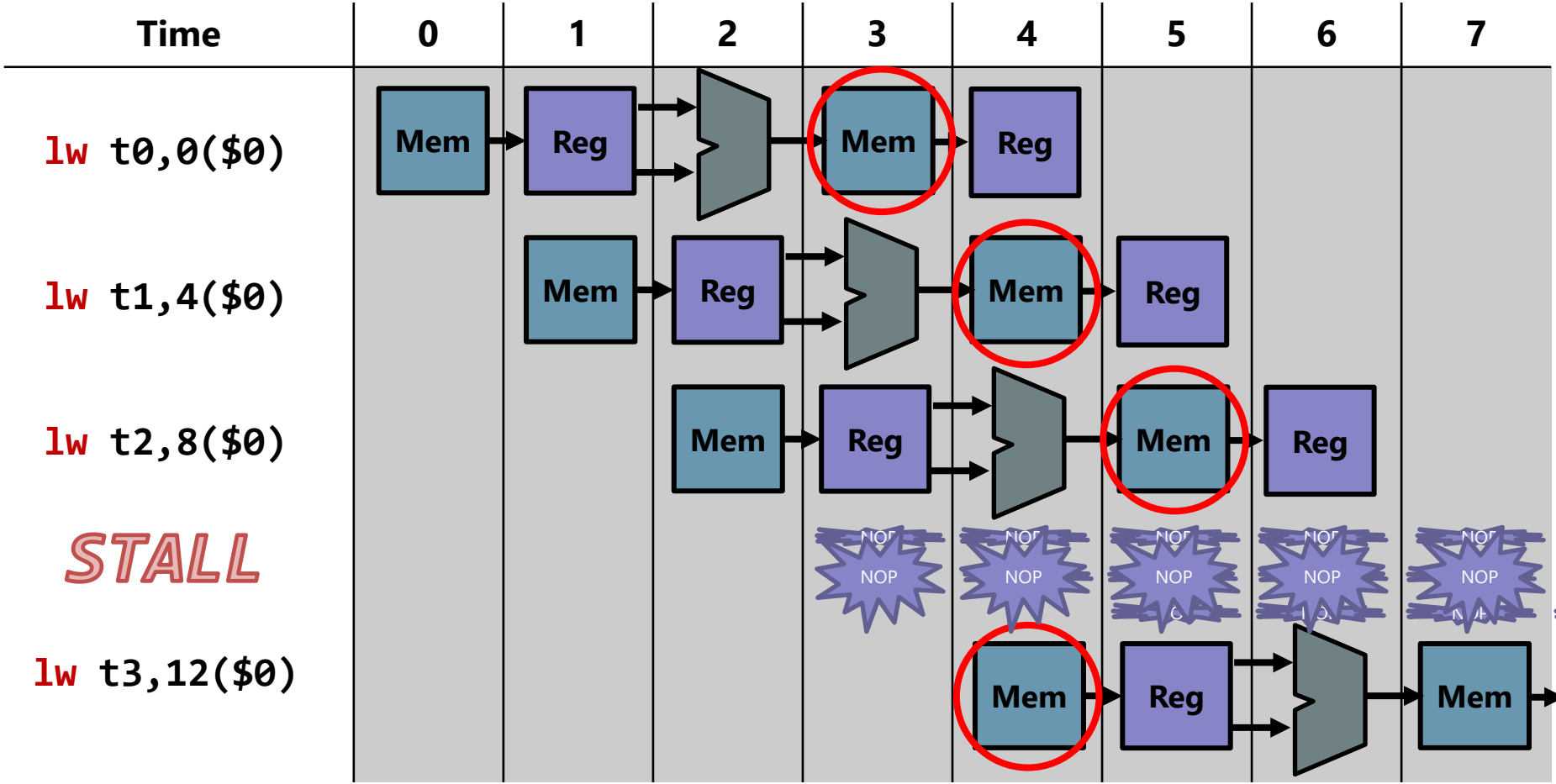
Structural hazards – What can I do???

- Structural hazards arise from **lack of resources**
- So… We can **eliminate the hazard by adding more resources**!
  - Add a second memory?
    - The Harvard architecture!
- Another solution:
  - Stall the instruction until the resource is available



Structural hazards – What can I do???

- You may need more than one **stall**!



Data hazards

Attempting to use an item before it is ready. E.g.:

- Only one sock of a pair is found during folding
- It's in the dryer! Folding has to wait!

In a CPU:

- Instruction depends on result of prior instruction still in the pipeline

```
add s0, t0, t1

sub t2, t2, s0
```



s0 must be produced before it can be used

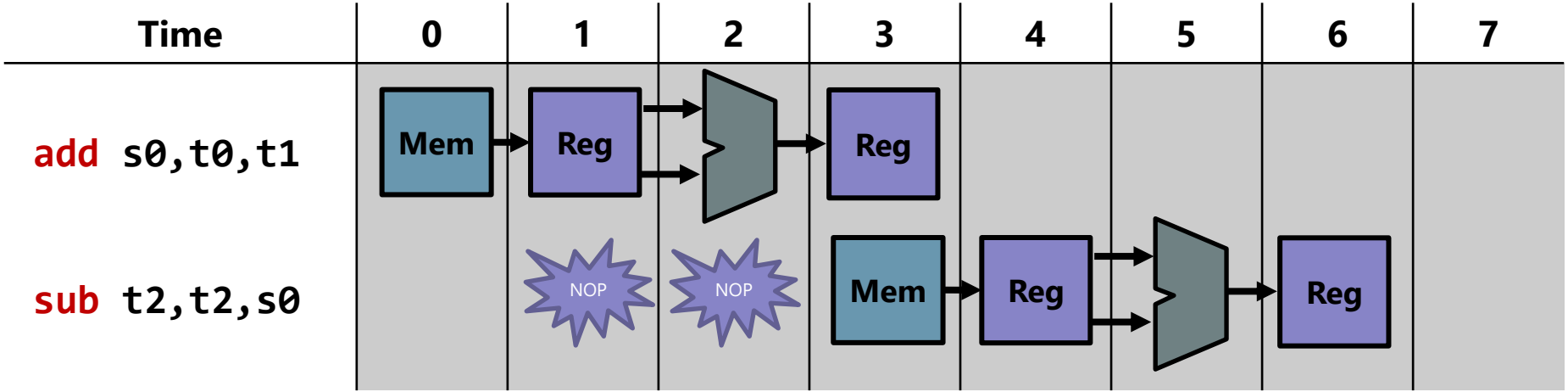
Data hazards – What can I do???

- Are these common?
  - **Yup**! You bet!

    i=i+1

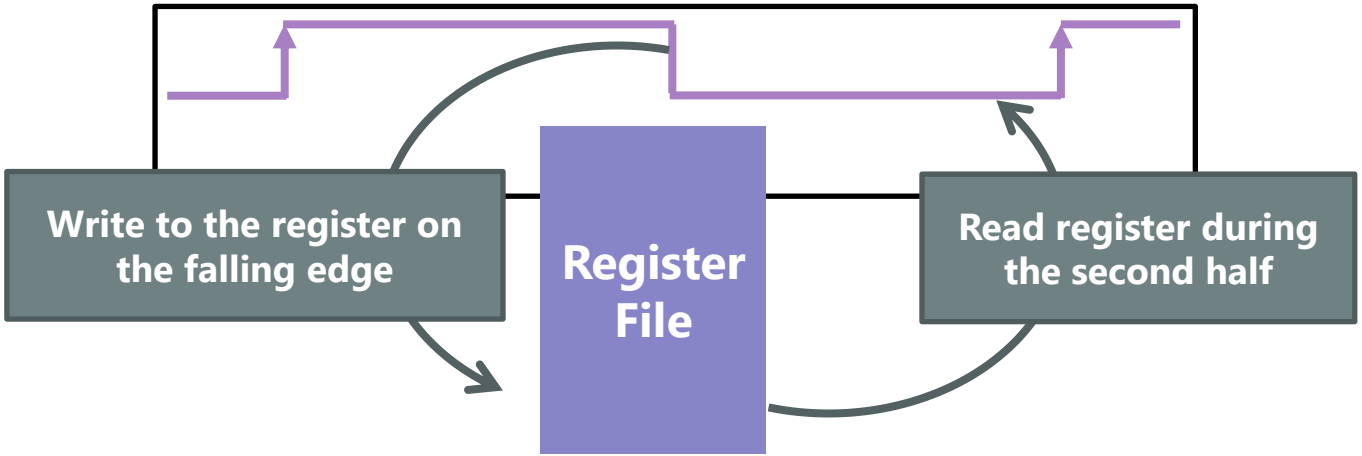
    array[i]

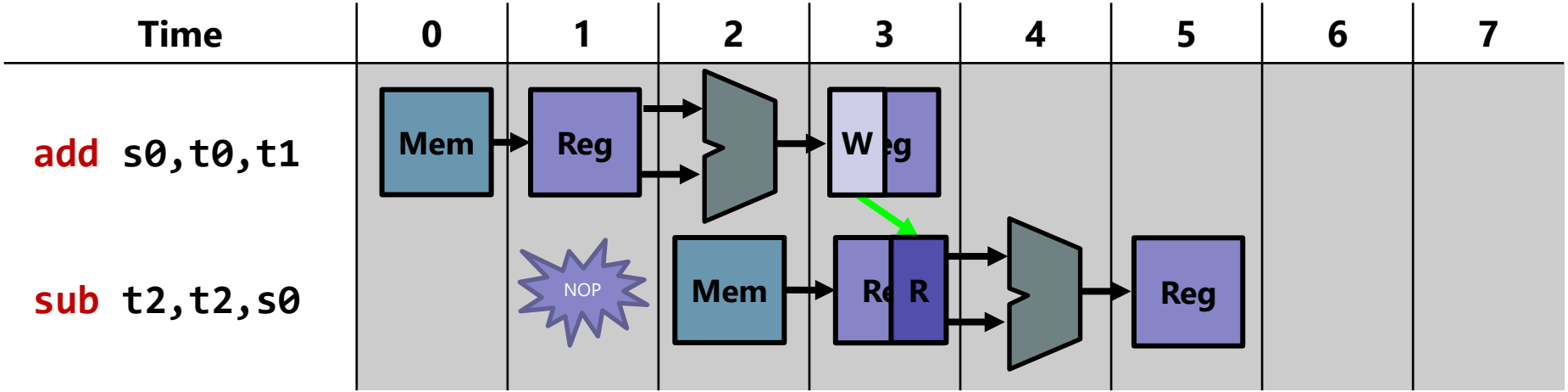
- Solution 1: Stall until value is written back to the register file
  - Penalty is high with this solution.



Data hazards – What can I do???

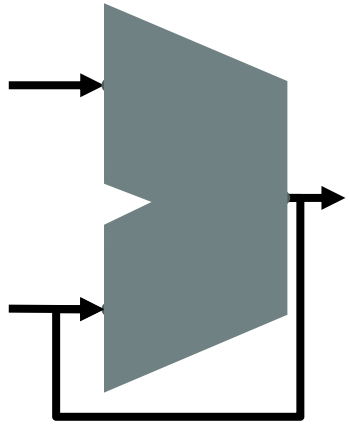
- Solution 2: What if we improve the register file?



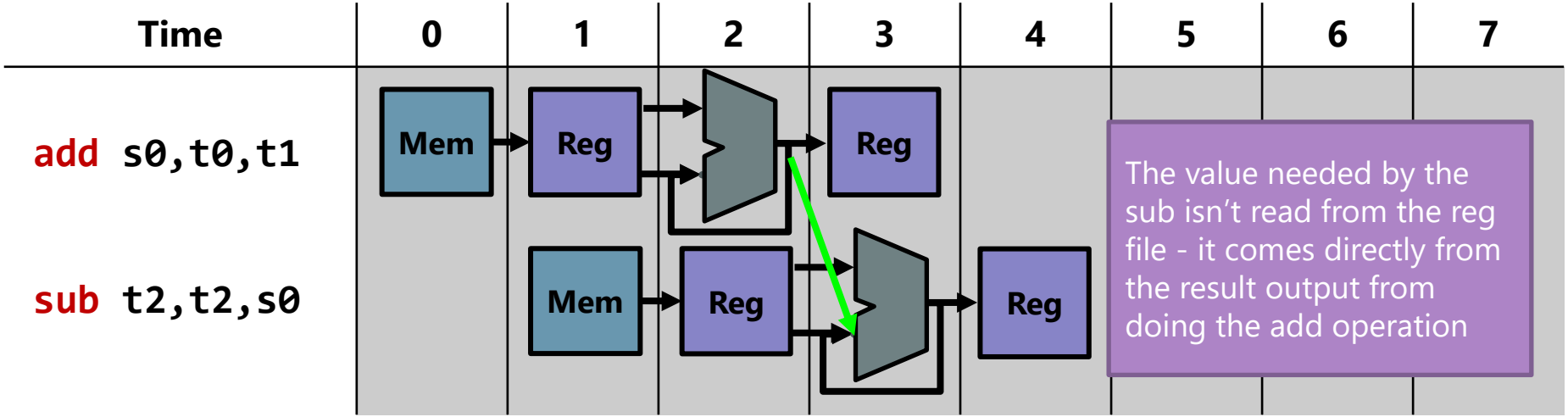


Data hazards – What can I do???

- Solution 3: Can we forward the ALU output?
  - Add path from ALU output to one of its inputs



Forwarding: Passing the result from a later stage to an earlier one



Control hazards

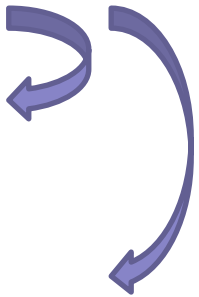
Attempting to make a decision before condition is evaluated

- If the dirty clothes are not clean after washing!
- Then I must wash them again

In a CPU:

- Branches

```
        blt s0, s1, DONE
        add t0, t1, t2
        or  t0, t1, t2
DONE:
        sub t0, t1, t2
```



Which path will the program take?

Which instruction do we fetch next?

Control hazards

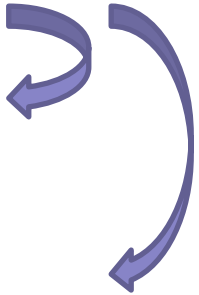
Attempting to make a decision before condition is evaluated

- If the dirty clothes are not clean after washing!
- Then I must wash them again

In a CPU:

- Branches

```
        if s0 < s1 goto DONE
        add t0, t1, t2
        or  t0, t1, t2
DONE:
        sub t0, t1, t2
```

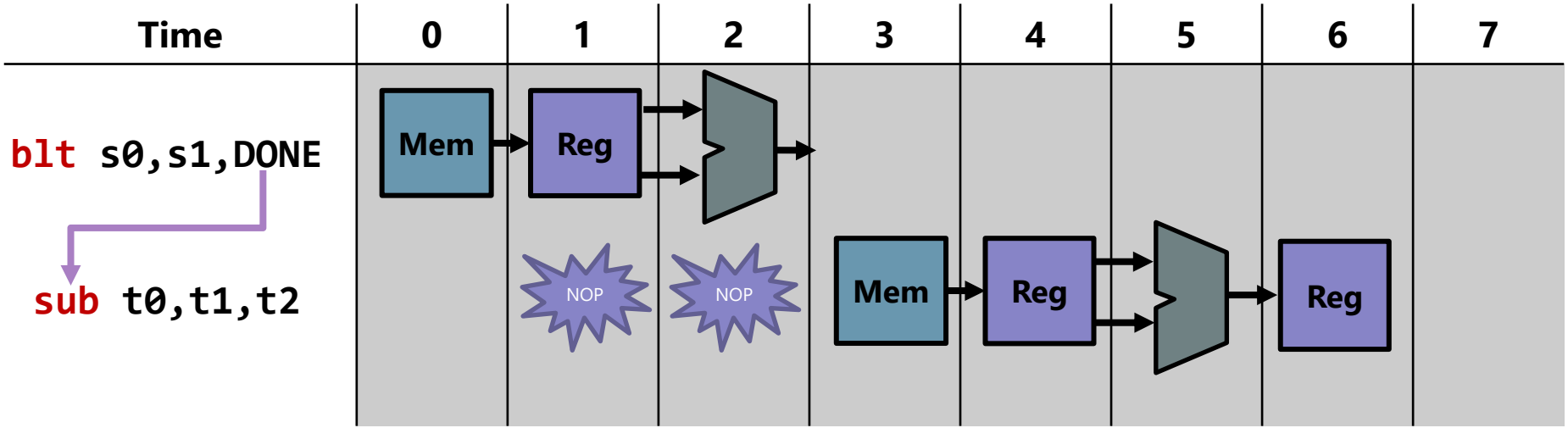


Which path will the program take?

Which instruction do we fetch next?

Control hazards – What can I do???

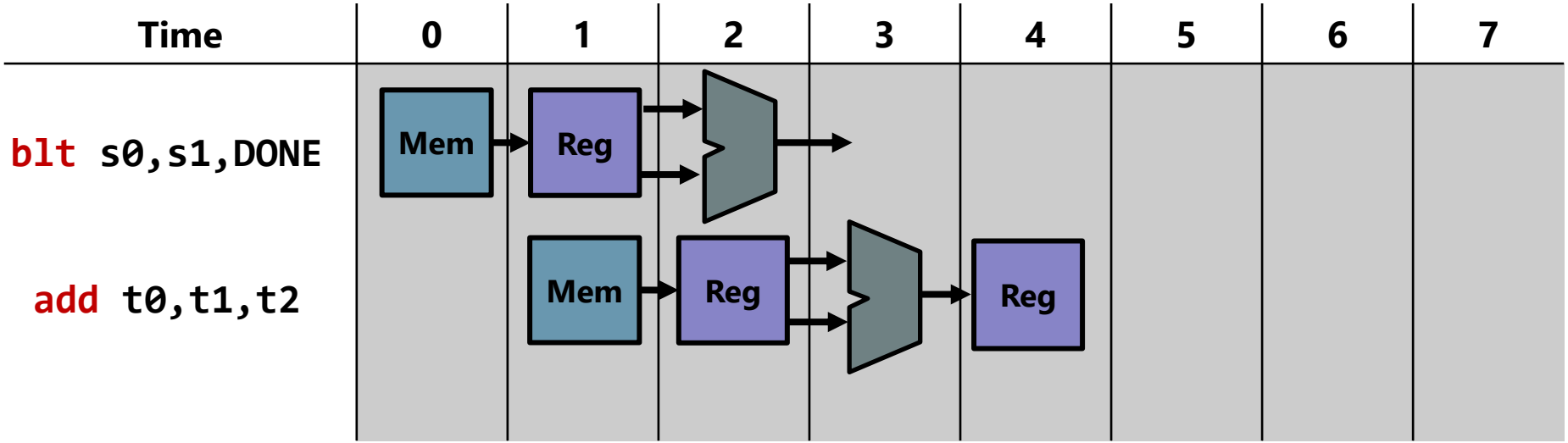
- **We can stall…** until the outcome is known!



- This is a bit wasteful! We really don't like stalls! ☺
- We want the pipeline always full and doing useful work!

Control hazards – What can I do???

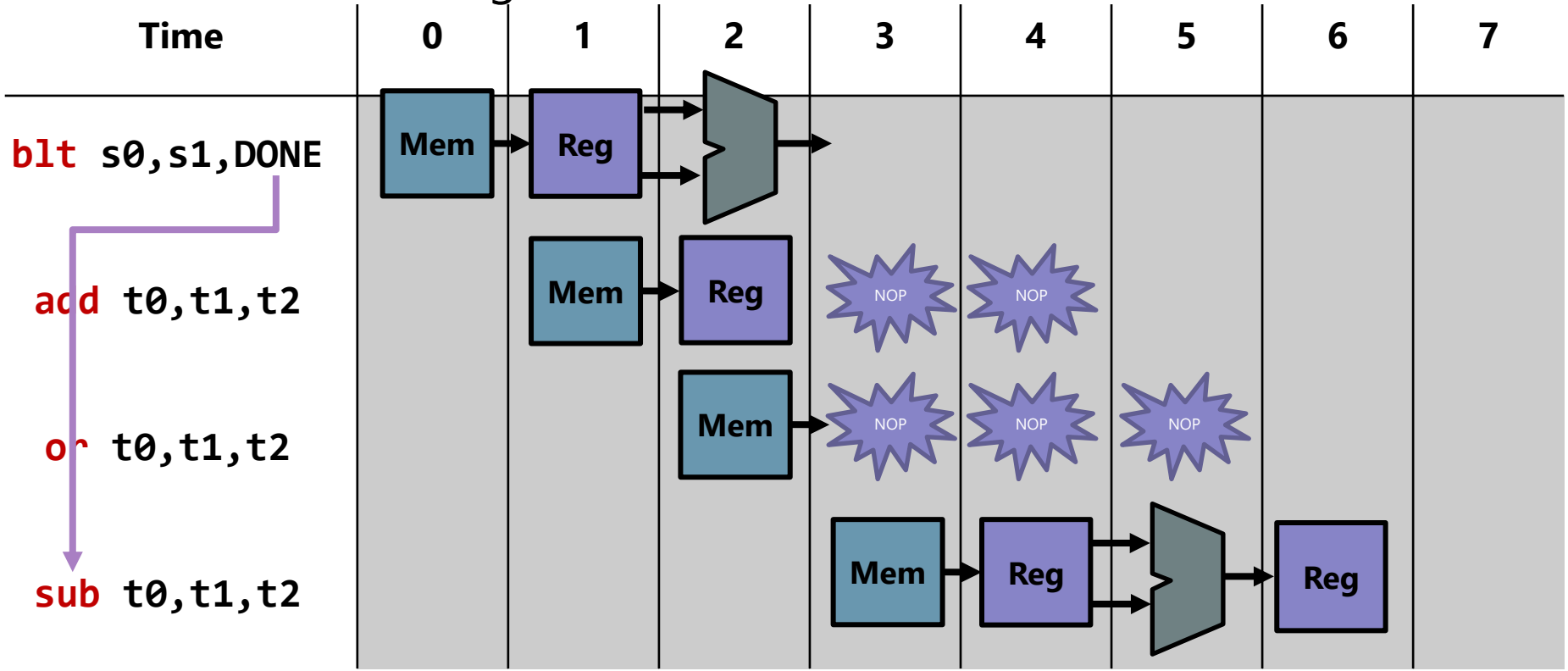
- **Sooo…** we can **predict** that the branch is never taken! (naïve)



- We attempt to execute the next sequential instruction!
- It is a **gamble!** that the branch will never be taken.
  - But **if we are right**, there is no **stall!!!** ☺

Control hazards – What if we are wrong?????!!!

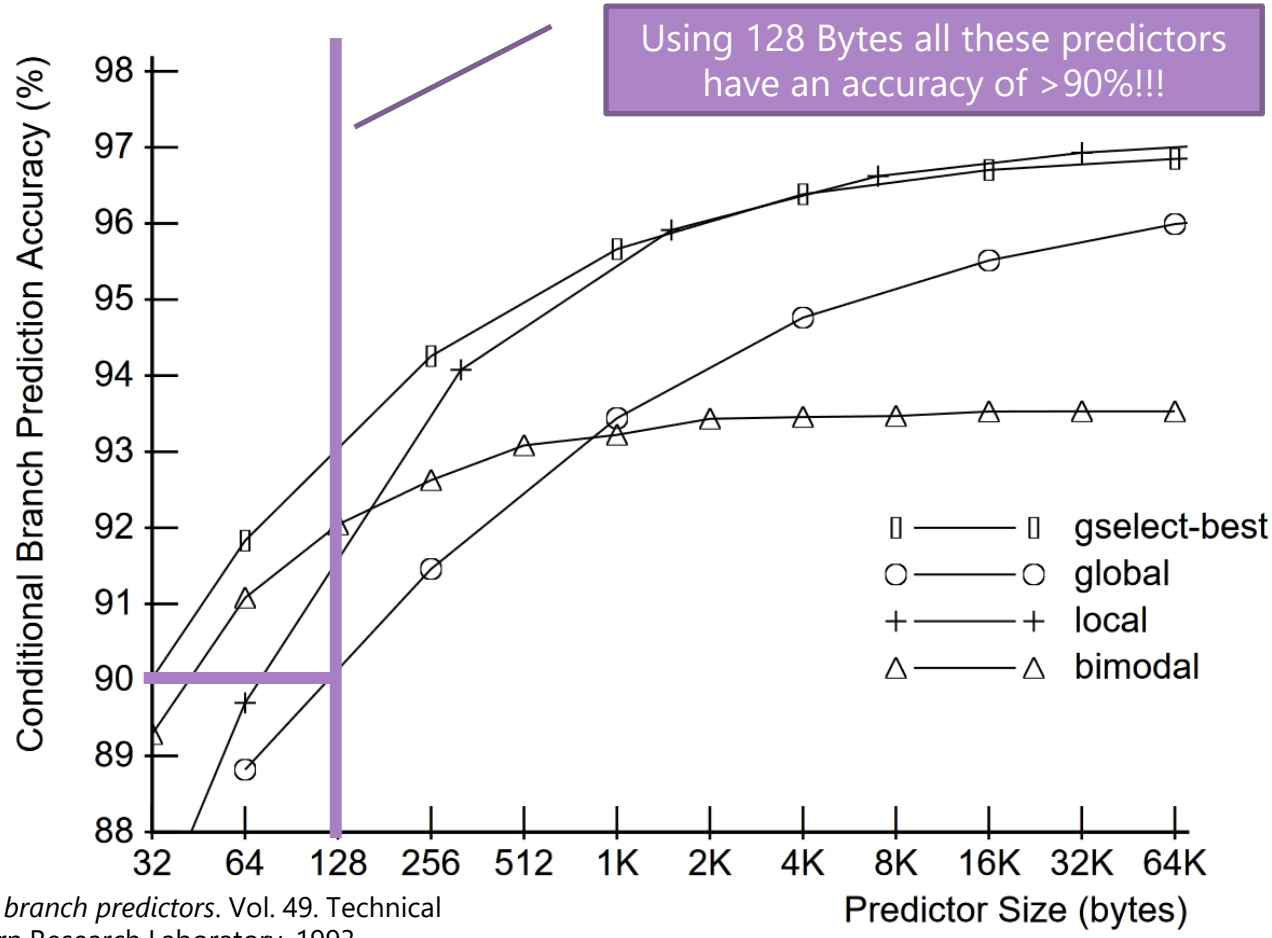
- **Ok,** what if we are wrong???!!



- Just abort (stall the remaining steps) to fix it! Nothing was actually changed!
- Read the correct instruction!

Fun facts!

- How often do you think a (less-naïve) branch predictor is correct?



McFarling, Scott. *Combining branch predictors*. Vol. 49. Technical Report TN-36, Digital Western Research Laboratory, 1993.

What to know more?

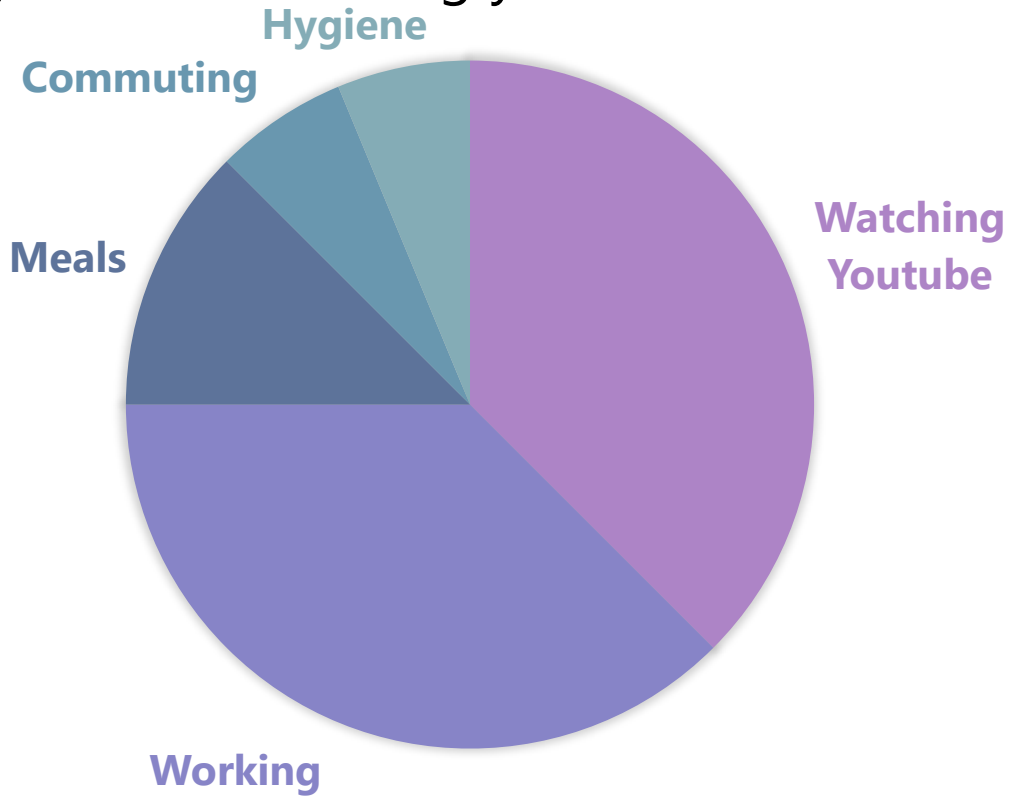
- CS 1541 – Introduction to Computer Architecture
  - Learn more about hazards.
  - Learn more about branch predictors.
  - Learn about memory hierarchies.
  - And more…

Performance and The Law of Diminishing Returns

Don't waste your time...

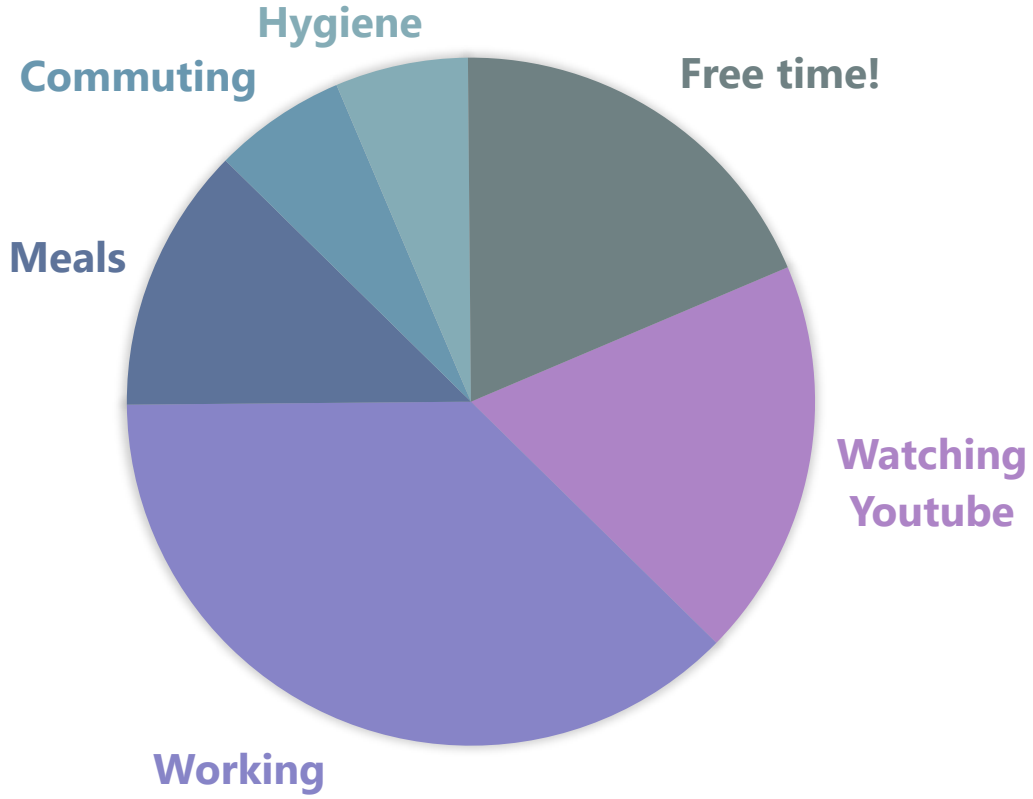
- suppose you're trying to get better at time management
- you got an app that lets you time how long you do stuff

if you wanted to get more free time by **halving the amount of time** it takes to do one task, which task would you choose?



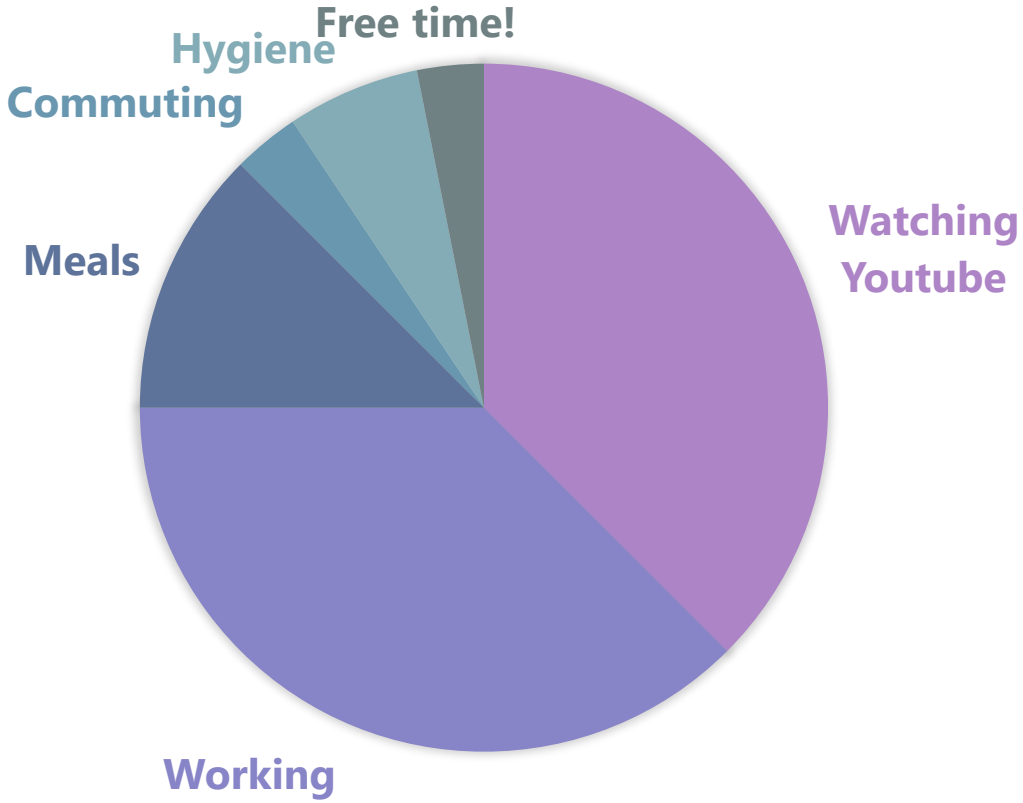
If you cut Youtube by half...

- look at all the extra free time you have!



If you cut commuting by half...

- look at ... all the extra free time... you have.



The tale of two multipliers

- The tale starts with a simple program

```
        li    $1, 100
L0:
        lw    $2, A[i]      ; pseudo-code to load A[i]
        lw    $3, B[i]      ; pseudo-code to load B[i]
        mult  $3, $2
        mflo  $4
        sw    $4, C[i]      ; pseudo-code to store C[i]
        addi  $1, $1, -1
        bne   $1, $0, L0
```

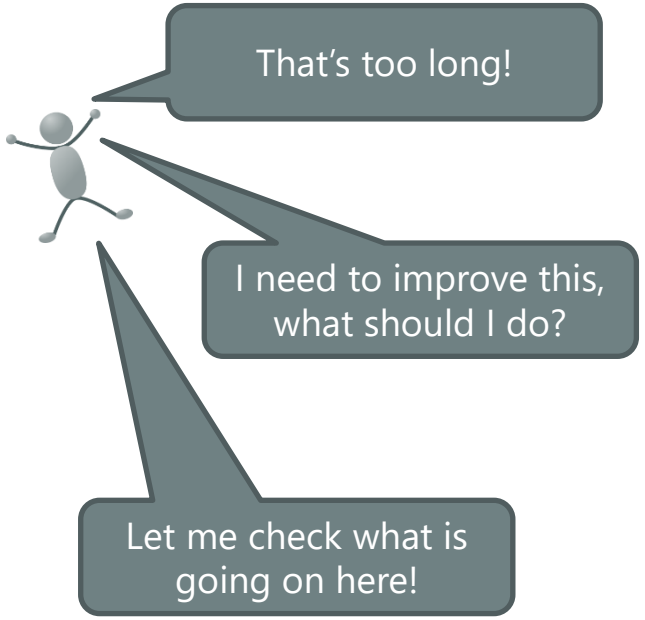
Runs 100 times

- How many times does the loop execute?

The tale of two multipliers

- You measure how long it takes to execute.
  - It took 102010ns

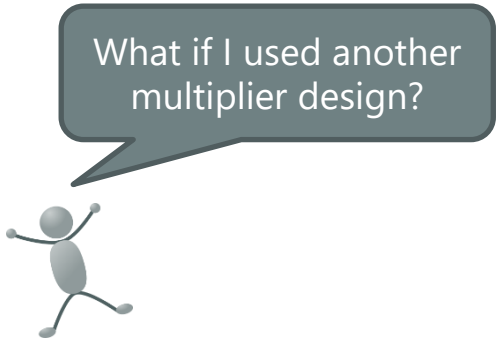
```
        li    $1, 100              10ns
L0:
        lw    $2, A[i]             10ns
        lw    $3, B[i]             10ns
        mult  $3, $2               960ns
        mflo  $4                   10ns
        sw    $4, C[i]             10ns
        addi  $1, $1, -1           10ns
        bne   $1, $0, L0           10ns
```



$$time = 1 \times 10ns + 6 \times 100 \times 10ns + 1 \times 100 \times 960ns = 102010ns$$

The tale of two multipliers

- It seems this CPU implements a slow multiplier
  - It needs to execute **3 distinct steps**:
    (1) add, (2) shift left, and (3) shift right
  - Multiplication takes 32-bit numbers
    - The ALU and the adder are 64-bits!
  - The 64-bit addition takes 10ns
  - Shifts also take 10ns
  - The multiplication takes 96 steps → 3 × 32bits
    - $\text{total} = 96 \text{ steps} \times 10ns = 960ns$



The tale of two multipliers

- I know, let's use a Fast multiplier design I have
  1. It combines some registers
  2. And we can make it do the **3 steps simultaneously**
  3. And the ALU only needs to be 32-bits!

  - Assuming a linear relationship between bits and adder speed:
    - The 32-bit addition takes 5ns

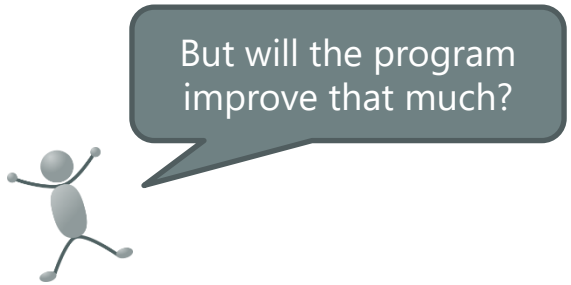
  - The multiplication takes 32 steps → $1(combined) \times 32$bits
    - $\text{total} = 32 \text{ steps} \times 5ns = 160ns$



The tale of two multipliers

- Let's calculate how much faster it is:
  - Let's calculate the **speedup** ratio:
    - The factor by which the new version is faster than the old one

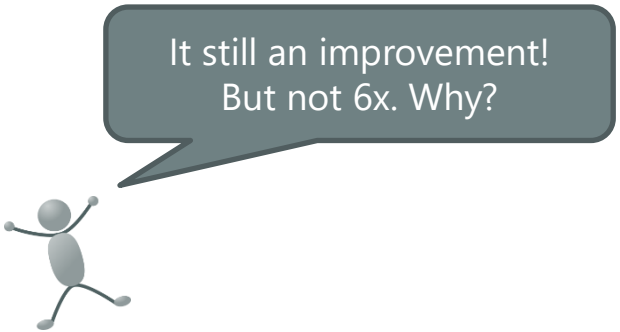
$$\text{speedup} = \frac{\text{slow multiplier time}}{fast\ multiplier\ time} = \frac{960ns}{160ns} = 6\times$$



The tale of two multipliers

$$time = 1 \times 10ns + 6 \times 100 \times 10ns + 1 \times 100 \times 160ns = 22010ns$$

$$\textbf{speedup} = \frac{\text{slow program}}{fast\ program} = \frac{102010ns}{22010ns} = \mathbf{4.63\times}$$



- The multiplier is only used once!

The tale of two multipliers

- What happens if we decrease the proportion of execution time?

Suppose 101 instructions: 100 non-multiply, 1 multiply

$$time\ slow = 100 \times 10ns + 960ns = 1960ns$$

$$time\ fast = 100 \times 10ns + 160ns = 1160ns$$

$$speedup = \frac{1960ns}{1160ns} = 1.7\times$$

Suppose 1001 instructions: 1000 non-multiply, 1 multiply

$$time\ slow = 1000 \times 10ns + 960ns = 10960ns$$

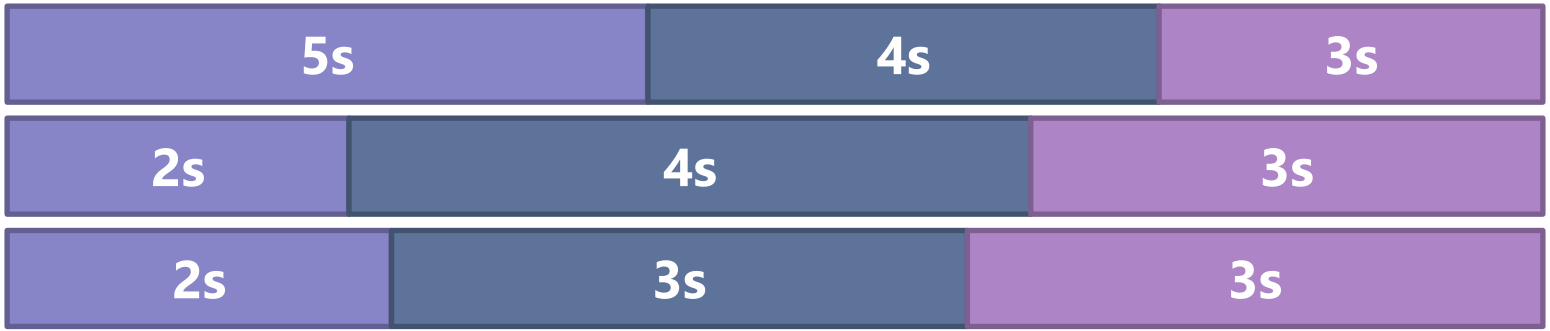
$$time\ fast = 1000 \times 10ns + 160ns = 10160ns$$

$$speedup = \frac{10960ns}{10160ns} = 1.08\times$$

Decreasing gains

- $6\times \rightarrow 4.63\times \rightarrow 1.7\times \rightarrow 1.08\times$
- What happened?
  - Proportion of time spent multiplying was not enough to have gains

- Optimization is a balancing act.
  - As you solve a bottleneck, a new one will appear.
  - Improve things to a point, then there are diminishing returns!



What about pipelining?

- How much faster (and why) is a pipelined implementation of MIPS?
- As we saw last class, we compute the speedup for this:

$$\text{speedup} = \frac{\text{slow time}}{fast\ time}$$

- And how do we compute "time"?

$$\text{CPU time} = n \times CPI \times t \text{ seconds}$$

Average Instruction CPI

- What is an "average instruction" **CPI**?
  - Remember how we calculated the average CPI of a program?

- Given a program, how many **cycles** does an instruction **typically** take?
  - It depends on the program!
  - How many instructions, and what types?
    - E.g.: all adds vs. all loads for multi-cycle implementation

- The average instruction CPI is the average cycle count per instruction

Instruction Mix

- **Instruction mix:** Is the % total instruction count (n) corresponding to each instruction class
- **Program A:** 100 adds, 100 subtracts, 50 loads, 25 stores, 50 branches, and 10 jumps. Total 335 instructions.

- What is the mix?

Arithmetic	(100+100) / 335 =	0.597 =	59.7%
Load	50 / 335 =	0.149 =	14.9%
Store	25/335 =	0.075 =	7.5%
Branch	50/335 =	0.149 =	14.9%
Jump	10/335 =	0.03 =	3.0%

CPI – Multi-cycle

- Given this mix, what is the Average Cycles Per Instruction (CPI)?
  - E.g., with a multi-cycle CPU.

- We compute the weighted average

$$\text{CPI} = \Sigma_{all\ classes}\ freq \times cycles$$

Class	Frequency	Cycles	Contribution
Arithmetic	59.7%	4	2.388
Load	14.9%	5	0.745
Store	7.5%	4	0.3
Branch	14.9%	3	0.447
Jump	3.0%	3	0.09
		Total	3.97 CPI

CPU time

- And now we can calculate the CPU time
  - Assuming a cycle length of 2ns

$$\text{CPU time} = 335 \times 3.97 \times 2ns$$
$$= 2660ns$$

Class	Frequency	Cycles	Contribution
Arithmetic	59.7%	4	2.388
Load	14.9%	5	0.745
Store	7.5%	4	0.3
Branch	14.9%	3	0.447
Jump	3.0%	3	0.09
		Total	3.97 CPI

What about in the pipeline implementation?

- In the best case, what is the CPI?
  - How many instructions are we starting every clock cycle?

- What about the typical case, what is the CPI?
  - We have to consider hazards.
  - Say, 20% of branches are predicted correctly.
  - 60% of loads do not conflict with other memory accesses.

- Assume the same program and clock cycle.

Instruction Mix – Pipeline

- Instruction mix: Treat the delayed load and branch instructions as a separate class

Class	Frequency	Cycles
Arithmetic	59.7%	1
Load – no delay	0.6*14.9%=8.94%	1
Load – delay	5.96%	2
Store	7.5%	1
Branch predicted	0.2*14.9%=2.98%	1
Branch not predicted	11.92%	3
Jump	3.0%	1

CPI – Pipeline

- We compute the weighted average

$$\text{CPI} = \Sigma_{all\ classes}\ freq \times cycles$$

Class	Frequency	Cycles	Contribution
Arithmetic	59.7%	1	0.597
Load – no delay	0.6*14.9%=8.94%	1	0.0894
Load – delay	5.96%	2	0.1192
Store	7.5%	1	.075
Branch predicted	0.2*14.9%=2.98%	1	.0298
Branch not predicted	11.92%	3	0.3576
Jump	3.0%	1	0.03

Total	1.30 CPI

The speedup

- Compute CPU execution time of pipelined implementation
  - Every value except CPI is the same as in the multi-cycle
    - n – is a property of the program

$$Pipeline\ CPU\ time = 335 \times 1.30 \times 2ns$$
$$= 871ns$$

$$\text{speedup} = \frac{2660ns}{871ns} = 3.05 \times$$